

Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) using adapter hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per preregistered rules). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test $p = 0.021484375$; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Execution used the declared model gpt-5-mini and recorded 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair test was not produced by the archived confirmatory summary artifacts; the preregistered harmful-change mapping is archived (see Methods) and the per-episode raw traces required to compute that preregistered statistic are available in the evidence bundle (execution/raw_results.jsonl and scoring traces). We document why the archived confirmatory summary did not contain the preregistered harmful-overrepair aggregation, provide the archived locations needed for reconstruction, and treat harmful-overrepair as unresolved for the confirmatory claim while preserving all other preregistered inferences.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A common safety-oriented architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are exposed to a generator/repair model to produce corrected candidates. Prior empirical work in code and Infrastructure-as-Code (IaC) settings reports verifier-interposed repair can raise verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and imposing interaction costs [1], [2], [3], [4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault

operators is sparse. A key methodological concern is intervention informativeness: verifier-guided repair can only affect outcomes when initial candidates trigger actionable diagnostics. We address that by running a preregistered controlled-fault injection (deterministic, outcome-independent) and a paired design that shares the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. RELATED WORK

Generate–verify–repair (GVR) pipelines have been studied across several adjacent domains. Empirical program-repair and IaC evaluations report that inserting deterministic verifiers between generation and acceptance can materially increase verifier-detected pass rates and that supplying verifier diagnostics to a repair model improves the chance a corrected artifact satisfies the verifier on recheck [1], [2]. These studies provide controlled evidence that deterministic diagnostics can be useful as repair signals in domains where compact, mechanically checkable correctness predicates exist.

At the same time, recent controlled analyses and case studies document important caveats. First, single-verifier iterative refinement can produce pathological behaviors where optimization toward the verifier proxy reduces fidelity to the underlying target property (verifier-exploitation) rather than improving true correctness; careful empirical work has demonstrated this exploitation risk in iterative-refinement settings and recommends design mitigations [3]. Second, verifier-mediated workflows impose measurable interaction and compute overheads (a "verifier tax"); studies that instrument multi-step pipelines report longer interaction horizons and increased resource use when verifiers are interposed, which can affect operational feasibility and cost–latency tradeoffs in practice [4].

Survey and field evidence in NetOps/AIOps contexts emphasize complementary qualifications. Large-scale surveys and demonstration-network case studies show that LLMs are often beneficial for read-oriented tasks (diagnosis, documentation, and suggestion), but they also highlight that read-oriented usefulness does not automatically translate into safe closed-loop

execution without strong verification and conservative orchestration [5]. Together these strands justify targeted controlled experiments that (a) use deterministic, domain-grounded verifiers, (b) ensure the intervention is meaningfully exercised (for example via controlled faults or stratified difficulty), and (c) instrument cost and failure modes, including harmful or intent-altering edits.

Limitations of the existing literature motivate our design choices. The strongest empirical GVR results in the supplied records concentrate on IaC, code, and DSL benchmarks; cross-domain validation for vendor CLIs and heterogeneous device behaviors is limited in the supplied evidence, leaving open questions about transferability, verifier coverage, and residual unsafe outcomes after repair [1], [2], [5]. The literature also underscores methodological pitfalls that we addressed in our preregistration: avoid uninformative interventions by using a deterministic controlled-fault challenge, adopt paired semantics that preserve shared source candidates where appropriate for the causal estimand, and predefine a harmful-change mapping and missingness rules so safety aggregations are auditable and reproducible.

III. METHODOLOGY

Overview and preregistration. We preregistered the full confirmatory design as `vCLI_fault_injection_repair_2026_A_prereg_v1` prior to observing any confirmatory outcomes. The preregistration specifies the primary estimand (`paired_success_rate_difference_guarded_minus_baseline`), a single predefined primary exact paired test, the controlled-challenge sampling plan (deterministic fault indices 1..40), inclusion/exclusion rules, the missingness sensitivity treatment, the harmful-change labeling schema used for the preregistered safety aggregation, and a replication-friendly analysis recipe. The execution manifest records the preregistration was present prior to confirmatory execution; the manifest and all artifacts are archived and cited in the evidence bundle.

Adapter semantics, pairing design, and controlled-challenge sampling. Confirmatory execution used the adapter family `hosted_netops_validator_feedback_repair_v2` under its `shared_controlled_fault_candidate` semantics. Per the archived contract and preregistration: for each task index the deterministic fault injector produced exactly one controlled-fault candidate; that exact candidate was reused by both paired conditions. Exactly one initial sampled generation per task pair was performed and shared between arms; subsequent repair opportunities were then executed according to the adapter’s repair-call budget. In our preregistered design each pair received at most two repair episodes (baseline and guarded), and the adapter-mandated confirmatory `task_count` was fixed at 40.

Model and runtime sampling semantics. The execution recorded the declared model as `gpt-5-mini` in `execution/model_configuration.json`. The archived model configuration shows `temperature` was `null/unset`. We note ex-

plicitly: `null/unset` is not evidence of deterministic sampling and does not imply `temperature=0`. Sampling non-determinism therefore remains a possible contributor to within-condition variability; however, because the initial source generation was sampled once and shared across both arms (`shared_controlled_fault_candidate` semantics), any cross-condition differences in the confirmatory estimand arise from the downstream diagnostic exposure and repair calls rather than from independent initial candidate sampling.

Controlled-fault construction and informativeness guarantee. The confirmatory bank was generated by a deterministic fault injector (task indices 1..40). Each injected candidate encoded realistic actionable violations (examples: dropped-required-restore sequences, protected-interface modifications, make-before-break uplink switchover patterns). The injector design and selection rule were frozen in the preregistration; the execution manifest records the generator-run summary (planned 40 source-generation attempts; `source_valid_count` = 39; `source_invalid_or_failed_count` = 1). The controlled-challenge approach ensures the intervention (verifier-guided diagnostics) had meaningful opportunities to be activated across the confirmatory sample because every pair began from a deterministically injected, potentially invalid candidate rather than relying on low naturalistic activation rates.

Repair budgets, stage accounting, and execution instrumentation. The preregistered adapter contract bounded total confirmatory model calls; runtime accounting is available in the execution manifest. Key observed execution tallies (archived): planned confirmatory pairs = 40; `completed_episode_count` = 78 (39 complete pairs, two failed episodes); hosted-model calls used = 118 ($\leq$ planned maximum 120). Stage-level counts recorded by the adapter (deterministic reconciliation) show `valid_source_generation` = 40, `blind_controlled_fault_repair` = 39, `validator_guided_controlled_fault_repair` = 39. These instrumented counters are archived and permit independent cost-instrumentation and verifier-tax analyses. The preregistration also froze the `maximum_repair_calls_per_task` and retrieval augmentation settings; those fields are recorded in the model and execution configuration artifacts.

Scoring, primary success predicate, and harmful-change mapping. The primary binary success predicate required agreement of (1) the deterministic vendor-CLI verifier and (2) the simulator-based compliance check; both components are deterministic in the archived scoring pipeline. The verifier used in scoring is the `deterministic_netops_validator` (versioned in the archived scoring traces). The preregistration froze a harmful-change labeling schema intended to operationalize a confirmatory safety hypothesis: for the preregistered harmful-overrepair aggregation we labeled any per-episode post-repair validator violation that included either `PROTECTED_INTERFACE_CHANGE` or `UNINTENDED_STATE_CHANGE` as a harmful change. This compact operational mapping (the exact violation-code $\rightarrow$ harmful-change rule used by the archived scoring/transformation pipeline) is reproduced here: `harmful_change_indicator`

= ("PROTECTED_INTERFACE_CHANGE" in validation_after.violation_codes) OR ("UN-INTENDED_STATE_CHANGE" in validation_after.violation_codes). The executable transformation artifact implementing that mapping is archived as a transformation in the evidence bundle; its location within the archived execution manifest is execution/transformation_manifest.json (see evidence bundle for the exact artifact file). Per-episode scoring traces include the validator_trace.violation_codes and validation_after.violation_codes fields required to reproduce the preregistered harmful-overrepair counts.

Missingness, exclusions, and sensitivity rules. The preregistration specified an exclusion rule 'complete_pair_primary': exclude a pair from the primary analysis only if both paired condition executions are unscorable for infrastructure reasons. Execution produced one such excluded pair (source generation invalid, producing two unscorable condition episodes). The primary analysis therefore used complete_pairs = 39 and reports the number and causes of exclusions. The preregistration further preregistered a sensitivity analysis that treats missing/unscorable episodes as failures; that sensitivity recipe is implemented by the archived analysis executors and is reproducible from the raw artifacts (analysis/missingness_summary.csv is archived).

Statistical analysis recipe and reproducibility parameters. The preregistered primary test is an exact paired binary test (two-sided McNemar exact or equivalent exact paired binomial test) on the combined success indicator (verifier+simulator). The preregistration required reporting discordant-pair counts, absolute paired risk difference, two-sided exact p-value, and a 95% paired-bootstrap percentile confidence interval. The archived confirmatory analysis used bootstrap_resamples = 10000 and bootstrap_seed = 1729 for the CI computations; those parameters and the analysis outputs (paired contingency table and derived CSVs) are archived (analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis directory hashes recorded in the execution manifest). All raw per-episode records and provider traces (execution/raw_results.jsonl and execution/provider_calls/*.json) are archived to allow independent recomputation of the exact paired test and bootstrap CIs.

Reconstruction recipe for the preregistered harmful-overrepair statistic. Because the preregistered harmful-overrepair aggregation was defined in terms of validator post-repair violation codes and the transformation artifact implementing the mapping is archived, independent reproduction is possible without additional model calls. To reproduce the preregistered confirmatory harmful-overrepair counts from the archived artifacts an auditor should: (1) iterate per-episode records in execution/scoring/*.json or execution/raw_results.jsonl for the confirmatory episode set (task indices 1..40); (2) for each completed episode extract validation_after.violation_codes; (3) apply the compact mapping harmful_change_indicator = ("PROTECTED_INTERFACE_CHANGE"

in validation_after.violation_codes) OR ("UN-INTENDED_STATE_CHANGE" in validation_after.violation_codes) to produce per-episode harmful flags; (4) aggregate harmful_flag counts by condition across the confirmed complete pairs only (respecting the preregistered exclusion rule); and (5) run the preregistered paired comparison (paired difference in harmful proportions, exact paired test, and bootstrap CI using the archived bootstrap seed/resamples if desired). The precise transformation artifact implementing the mapping and its archived location are recorded in execution/transformation_manifest.json in the evidence bundle; the raw per-episode traces required for the mapping are in execution/scoring/*.json and execution/raw_results.jsonl. The preregistered analysis plan explicitly labeled this harmful-overrepair aggregation as confirmatory; the archived confirmatory analysis executor produced the primary paired result but did not emit the harmful-overrepair aggregation artifact. The reconstruction recipe above uses only archived artifacts and therefore does not require new model calls.

Implementation and artifact provenance. All runnable code for the data-processing, analysis, and scoring pipelines, plus the provider-trace collection code, was archived by the execution environment. Key archived artifact locations (refer to the evidence bundle for file-level hashes and the complete manifest): execution/raw_results.jsonl (raw per-episode records), execution/scoring/*.json (per-episode scoring/traces with validator_trace and validation_after), execution/model_configuration.json (declared model settings; temperature null/unset), execution/transformation_manifest.json (transformation artifacts including harmful-change mapping), analysis/*.csv (archived analysis outputs). The execution manifest contains the full artifact hash manifest and authoritative locations for reproduction. We avoid embedding full cryptographic digests in the prose; the evidence bundle contains the manifest for auditors.

Operational instrumentation and exploratory measures. The preregistration included exploratory secondary estimands (per-fault-class paired differences, model_call_cost_per_avoided_failure). The execution recorded hosted-model call counts (hosted-model calls used) and per-stage instrumentation (valid_source_generation, blind_controlled_fault_repair, validator_guided_controlled_fault_repair) that permit exploratory post hoc computation of model-call costs and per-outcome cost-per-avoided-failure; these instrumented counters are archived as part of the execution manifest and provider-call traces. Such exploratory cost analyses are reported as descriptive artifacts in the evidence bundle and are explicitly labeled exploratory in the preregistration.

Deviations, runtime checks, and stopping rules. The preregistered stopping rule required aborting the confirmatory run if the adapter contract could not be satisfied; no such silent substitution or stop-for-incompatibility occurred. Runtime deterministic reconciliation and provider-call audits verify that the adapter semantics were honored (one shared initial generation

per task pair, matched repair budgets, and no cross-condition cache reuse). The deterministic reconciliation and provider-call audit artifacts are archived and referenced in the evidence bundle for auditability.

Threats to validity related to methods. We emphasize the following methodological caveats grounded in the archived design: (1) sampling nondeterminism: temperature was null/unset in the archived model configuration and is not evidence of deterministic sampling; however pairing (shared initial generation) removes initial-generation sampling as a cross-condition confound. (2) verifier and simulator fidelity: the primary success predicate required agreement of two deterministic artifacts (verifier and simulator) whose coverage and fidelity constrain construct validity. (3) synthetic controlled-challenge bank: deliberate injection increases intervention activation at the cost of naturalistic representativeness. (4) single-model confirmatory execution: the declared model was fixed; broader model generality was not tested confirmatorily. We document these threats and provide the archived artifacts needed for transparent reanalysis and extension.

IV. RESULTS

Execution accounting. Planned confirmatory pairs = 40; completed episodes = 78 (39 complete pairs) with failed_episode_count = 2. Hosted-model calls used = 118 ($\leq$ planned maximum 120). Source-generation attempts = 40 (source_valid_count = 39; source_invalid_or_failed_count = 1). Run timestamps (UTC) are recorded in the execution manifest (started_at_utc = 2026-09-04T21:16:50.730326+00:00; completed_at_utc = 2026-09-04T21:19:34.370550+00:00). The analysis artifacts analysis/condition_summary.csv and analysis/paired_contingency_table.csv are archived with hashes in the evidence bundle.

Primary confirmatory outcomes (complete_pairs = 39). Baseline (blind repair) success = 30/39 = 0.7692307692307693. Guarded (verifier-guided) success = 38/39 = 0.9743589743589743. Paired contingency counts (guarded, baseline): n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. Paired estimate (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). Exact two-sided paired test (McNemar exact): $p = 0.021484375$. 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap_resamples = 10000, bootstrap_seed = 1729). The paired table and supporting CSV are archived at analysis/paired_contingency_table.csv (SHA-256 in analysis artifact manifest).

Missingness and excluded pair. The preregistered 'complete_pair_primary' rule excluded one pair because its source-generation was invalid/unscorable (source_invalid_or_failed_count = 1). That invalid source-generation produced two unscorable condition episodes (both_missing_pairs = 1). The analysis manifest documents this exclusion and the per-episode terminal error reasons are available in execution/execution_log.jsonl and execution/raw_results.jsonl for auditors.

Preregistered harmful-overrepair confirmatory statistic: status and reproducibility path. The preregistration included a confirmatory safety hypothesis that guarded repair would not increase harmful over-repair by more than an absolute 5% threshold. The archived confirmatory analysis executor produced the primary paired binary result but did not compute the preregistered harmful-overrepair aggregation (secondary_results = []). The raw per-episode verifier traces and provider traces required to compute the preregistered harmful-overrepair counts are archived (execution/raw_results.jsonl; execution/scoring/*.json). The preregistered harmful-change mapping (the frozen rules mapping validator violation codes to the harmful-change indicator) is archived as an execution transformation artifact; its manifest location is execution/transformation_manifest.json and the execution manifest includes the full artifact hash manifest (full_hash_manifest_path = execution/execution_manifest.json) enabling independent reproduction. Reconstructing the preregistered confirmatory harmful-overrepair statistic requires re-running the archived summary step that maps per-episode validator violation codes to the preregistered harmful-change label. To preserve the integrity of the archived confirmatory summary and because the archived confirmatory executor did not produce that preregistered summary artifact, we do not retroactively reaggregate and present a new preregistered confirmatory harmful-overrepair claim in this paper; instead we (a) document the omission, (b) provide the precise artifact locations and the compact operational mapping used by the archived scoring pipeline (PROTECTED_INTERFACE_CHANGE and UNINTENDED_STATE_CHANGE map to harmful-change), and (c) label any future counts computed outside the archived confirmatory summary as exploratory. The per-episode traces and the transformation manifest required for automated reproduction are archived and their paths are given above.

Representative diagnostic examples. Representative per-pair JSON scoring artifacts are archived (execution/scoring/task-000001-*.json, task-000002-*.json, task-000003-*.json, etc.). For example, pair-000001 shows a shared controlled-fault candidate corresponding to a 'dropped_required_restore' pattern; baseline and guarded repaired outputs are both valid after repair, and validator_feedback_exposed_to_model = true in the guarded episode trace. Pair-000003 illustrates a hard protected-interface change: the baseline repair left PROTECTED_INTERFACE_CHANGE violations after repair (validation_after.violation_codes includes PROTECTED_INTERFACE_CHANGE), while the guarded repair removed those violations and produced a valid configuration; these per-episode diagnostic traces are archived and auditable.

Secondary/exploratory analyses. The preregistered plan listed several exploratory items (per-fault-class descriptives, model_call_cost_per_avoided_failure). The archived execution accounting contains hosted-model call counts (hosted_model_call_event_count = 118) and stage counts (valid_source_generation = 40; blind_controlled_fault_repair

= 39; validator_guided_controlled_fault_repair = 39) allowing cost-per-outcome computations in exploratory follow-up. We report these exploratory analyses only as descriptive artifacts in the evidence bundle and label them explicitly as exploratory.

V. DISCUSSION

Interpretation. Under the preregistered controlled-challenge and shared-candidate pairing semantics, exposing deterministic verifier diagnostics substantially increased verifier+simulator-validated configuration success in our synthetic vendor-CLI task bank (approx. +20.51 percentage points). The shared-candidate paired design isolates the causal effect of diagnostic exposure because each arm received the same controlled-fault candidate, identical repair budgets, and identical model identity.

Boundaries and operational implications. (1) Confirmatory scope — The result applies to the preregistered synthetic controlled-fault bank, the declared model (gpt-5-mini), and the deterministic verifier+simulator used here; external generalization requires multi-model and multi-verifier replications. (2) Safety verdict unresolved — Because the archived confirmatory summary did not include the preregistered harmful-overrepair aggregation, we make no confirmatory safety verdict on the preregistered $\leq 5\%$ harmful-overrepair threshold; the archived per-episode traces and transformation manifest permit independent reconstruction. (3) Verifier tax and deployment feasibility — Hosted-model call counts and stage-level instrumentation are archived; quantifying large-scale operational cost/latency tradeoffs (the verifier tax) requires larger deployments and are outside the confirmatory scope. (4) Verifier-exploitation risk — Single-verifier iterative loops can be gamed in longer-horizon agentic pipelines; mitigation strategies include multi-verifier ensembles, calibrated abstention, and uncertainty quantification.

Reproducibility and auditability. All artifacts produced by the execution and analysis are archived in the evidence bundle. Key artifact locations: execution/raw_results.jsonl (raw per-episode records; SHA-256 recorded in execution manifest), execution/scoring/*.json (per-episode scoring traces), analysis/paired_contingency_table.csv and analysis/condition_summary.csv (archived analysis outputs), execution/model_configuration.json (declared model settings; temperature = null), and execution/transformation_manifest.json (transformation artifacts including the harmful-change mapping). The execution manifest contains the full hash manifest path (execution/execution_manifest.json) and representative artifact hashes cited in this manuscript. The exact initial master prompt is archived at provenance/master_prompt.txt (SHA-256: f781dd7978328198146d586cf33e0f4b783c8db126bd0f16fcd13699455ebb10) and is referenced in the Disclosure Statement.

Threats to validity. Internal validity — prompt sensitivity, sampling nondeterminism (temperature = null/unset), and the shared_controlled_fault_candidate semantics constrain sampling interpretations: exactly one sampled initial generation per pair was performed and reused; differences between

arms arise from diagnostic exposure and subsequent repair calls. Construct validity — primary success required both deterministic validator and simulator agreement; simulator fidelity limits construct validity. External validity — single model and synthetic task-bank limit generality to production heterogeneity. Conclusion validity — $n = 39$ complete pairs yields detectable effects of the observed magnitude but limits precision for fault-class subgroup inference. These threats are documented and linked to archived artifacts for transparent audit.

VI. LIMITATIONS

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: complete_pairs = 39 provides detectable effects of the observed magnitude but limits precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.
- Operational cost/latency unresolved for deployment: execution was instrumented and costs recorded, but large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.
- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and transformation manifest required to compute it are archived and available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. CONCLUSION

In a preregistered controlled-challenge paired experiment using hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; $p = 0.021484375$; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini),

and the archived deterministic verifier and simulator. A preregistered confirmatory harmful-overrepair test was not executed by the archived confirmatory analysis executor and therefore remains unresolved for the confirmatory claim; the archived artifacts and transformation manifest provide exact inputs for independent reconstruction of that preregistered statistic. We release the artifact bundle for reproducible reaggregation and recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

DISCLOSURE STATEMENT

Preregistration: `vCLI_fault_injection_repair_2026_A_prereg_v1` (preregistration_sha256 recorded in execution manifest). Adapter: `hosted_netops_validator_feedback_repair_v2`. Declared model used for confirmatory execution: `gpt-5-mini` (declared_model_version recorded in execution/model_configuration.json). Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256: `f781dd7978328198146d586cf33e0f4b783c8db126bd0f16fcd13699455ebb10`) and cited here by immutable archive reference. Human role: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: execution manifest records `temperature = null/unset` in execution/model_configuration.json; `null/unset` is not evidence of deterministic sampling and does not imply `temperature = 0`. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under `shared_controlled_fault_candidate` semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. Harmful-change mapping and reconstructability: the preregistered harmful-change rules were frozen and the transformation artifact implementing the mapping is archived (see execution/transformation_manifest.json and the full artifact manifest execution/execution_manifest.json in the evidence bundle). Per-episode raw traces and scoring artifacts required to reproduce all aggregated confirmatory and preregistered secondary statistics are archived (execution/raw_results.jsonl; execution/scoring/*.json; analysis/*.csv). The study followed the preregistered stopping rule; no silent adapter substitution occurred.

REFERENCES

- [1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [2] <https://arxiv.org/abs/2607.20478>
- [3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1
- [4] <http://arxiv.org/abs/2603.19328>
- [5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729